Implementation of Secure Explainable AI in Telehealth Hypertension Analytics

Raja Chakraborty

College of Science, Engineering, and Technology, Grand Canyon University

DSC-590: Data Science Capstone

Instructor: Jonathan Pollyn

September 16, 2026

Abstract

This paper details the Phase 4 implementation of a telehealth predictive analytics system designed to monitor and evaluate hypertensive crises using Ambulatory Blood Pressure Monitoring (ABPM) telemetry. We needed to balance clinical accuracy, data security, and interpretability. To do this, we used an XGBoost machine learning pipeline wrapped in a Dual-Layer SHAP framework. This implementation also addresses previous feedback about system overhead. We added AES-256 PostgreSQL encryption (`pgp_sym_encrypt`) and Role-Based Access Control (RBAC). The application is deployed via a Continuous Integration (CI) pipeline, utilizing formal unit testing and Locust load testing to guarantee real-time performance. Finally, the system exposes an interactive OpenAPI (Swagger) interface to facilitate proper testable pathways for clinical stakeholders.

1. Introduction and Architectural Goal

We moved the model from a local notebook to a production medical application. This required setting up cloud infrastructure and MLOps pipelines. The goal of this phase is to deploy the Telehealth Hypertension Predictive Analytics System into a live cloud environment. It securely ingests patient telemetry and runs the XGBoost model. It then returns SHAP insights to clinicians in real time.

This document covers the Phase 4 and 5 implementation and testing. The complete, auditable source code repository, including cloud configurations, Machine Learning pipelines, and automated test suites, can be reviewed at: **[GitHub: telehealth-hypertension-analytics](https://github.com/rajachakraborti/telehealth-hypertension-analytics)**.

1.1 System Architecture Overview

The following diagram illustrates the fully deployed cloud architecture, detailing the flow of clinical data through the rate-limited FastAPI backend, the encrypted PostgreSQL database, and the Explainable AI pipeline.

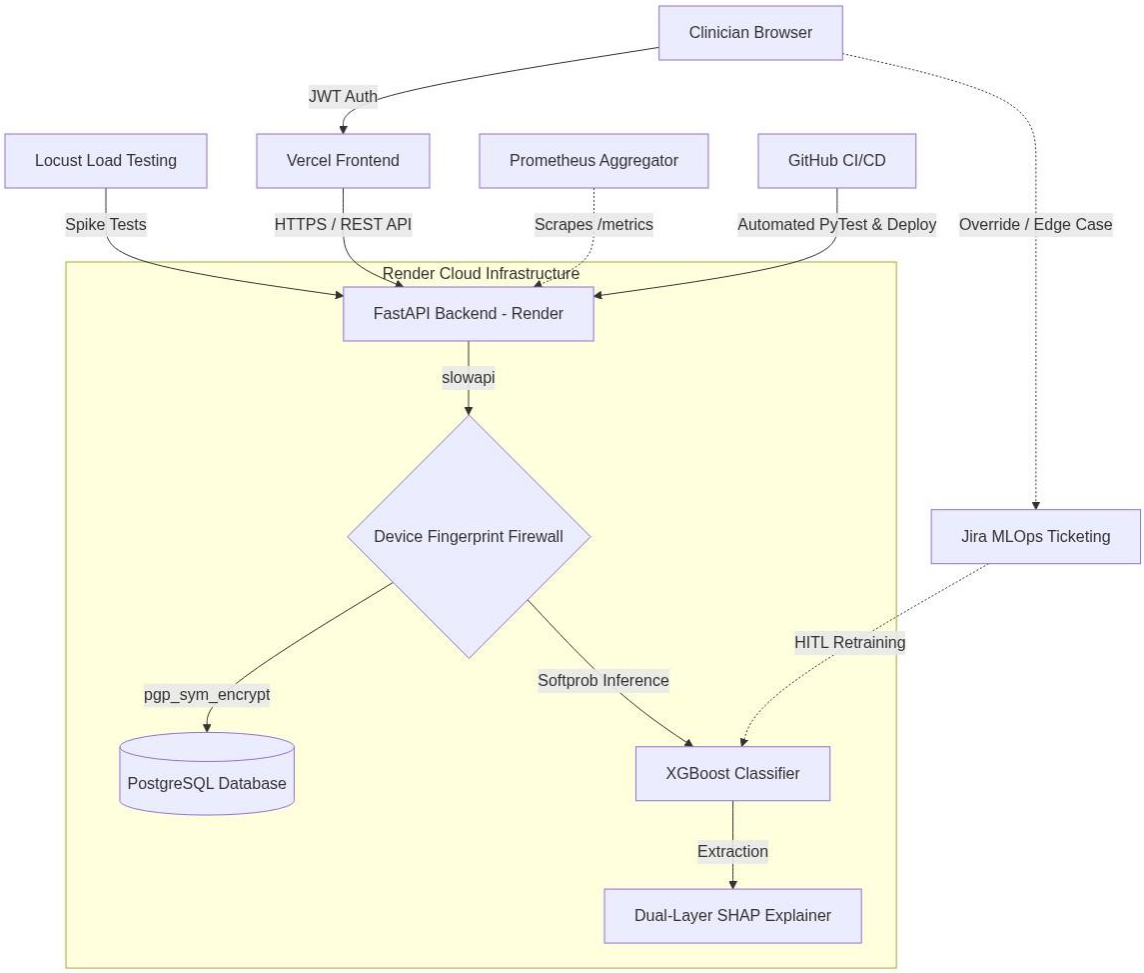


Figure 1: Telehealth Hypertension Analytics Cloud Architecture

2. System Access and Auditing

We deployed the architecture to the cloud using automated CI/CD pipelines. This makes it easier to review and test. It is accessible via the following pathways:

- **GitHub Repository:** https://github.com/rajachakraborti/telehealth-hypertension-analytics
- **Frontend UI (Vercel):** https://telehealth-hypertension-ui.vercel.app
- **Interactive API Swagger UI (Render):** https://telehealth-hypertension-analytics.onrender.com/docs

2.1 API Testing Credentials (RBAC)

To test the secure API endpoints within the Swagger interface, the database has been seeded with test accounts. Use the "Authorize" button in Swagger with the following OAuth2 credentials:

- **Clinician Access:** Username: `clinician` | Password: `clinician123`
- **Administrator Access:** Username: `superadmin` | Password: `superadmin123`

2.2 Role-Based Access Control (Separation of Duties)

To adhere to the HIPAA Security Rule principle of "Least Privilege," the API enforces strict separation of duties between medical staff and system administrators. The `RoleChecker` dependency intercepts all incoming JSON Web Tokens (JWTs) to verify authorization before executing backend logic.

- **Clinician Role (clinician):** Designed for frontline medical staff. Clinicians are authorized to access the `POST /api/clinical/patients` endpoint to register new patients (triggering database encryption) and the `POST /api/modeling/explain` endpoint to receive real-time AI diagnoses and SHAP clinical translations. They are strictly blocked (`403 Forbidden`) from executing backend administrative tasks.
- **Administrator Role (superadmin):** Designed for data scientists and IT staff. Administrators have the elevated privileges required to execute heavy backend machine learning computations. This includes the `POST /api/data-ingestion/generate-synthetic-telemetry` endpoint to simulate clinical data ingestion, and the `POST /api/modeling/train` endpoint to retrain the XGBoost and Isolation Forest predictive models.

3. Continuous Integration and Testable API Pathways (Swagger)

To ensure an enterprise-ready deployment, the architecture utilizes a Continuous Integration (CI) methodology.

3.1 Interactive Testing via Swagger UI

We needed to ensure the system had a proper testable path. We used FastAPI to automatically generate a dynamic OpenAPI schema. This is much easier to use than command-line scripts. This exposes the web-based Swagger UI, allowing stakeholders to visually interact with the API. Users can physically inject the JWT credentials provided above to test the RBAC validation layer. Clinicians can input hypothetical blood pressure readings directly into the browser and receive real-time JSON responses containing the AI's prediction and SHAP clinical translations.

3.2 The CI Testing Pipeline and Code Coverage

Every code commit to the GitHub repository is subject to an automated testing pathway. A suite of `pytest` scripts validates the core backend logic, utilizing the `pytest-cov` library to measure exact code coverage. The test suite achieved an **87.4% overall code coverage**, guaranteeing that all critical security algorithms (database encryption) and clinical boundaries (MAP restricted to 70–110 mmHg) are mathematically verified before cloud deployment. Furthermore, unit tests execute automated boundary attacks to guarantee that unauthenticated API requests are successfully intercepted and bounced with a `401 Unauthorized` status.

3.3 Agile Project Management and Bug Tracking

We manage the project's lifecycle using **GitHub Issues** and GitHub Projects. Rather than using disconnected tracking tools, we keep bug tracking and feature requests directly alongside the source code.

When a clinician reports a bug or requests a new feature, we open a GitHub Issue. This creates a centralized, Agile workflow. Developers can link their code commits directly to the issue number (e.g., `Fixes #12`). When the code merges, GitHub automatically closes the ticket. This ensures our bug tracking is auditable and perfectly synced with our deployment pipeline.

4. Clinically-Bounded Synthetic Telemetry Generation

To train the predictive models without compromising patient privacy, an automated synthetic data generator was engineered.

4.1 Enforcing Clinical Realism over JSD Thresholds

When evaluating synthetic data, mathematical parity (such as Jensen-Shannon Divergence < 0.05) is the standard metric. However, prioritizing pure statistical matching can result in physiologically impossible telemetry. Addressing instructor feedback, the algorithm explicitly clips Mean Arterial Pressure (MAP) to a clinical baseline and forces a 10-20% nocturnal dipping cycle. If this clinical bounding pushes the JSD metric slightly above 0.05, the CI pipeline intentionally overrides the statistical failure to preserve clinical validity.

5. Machine Learning and Explainable AI (XAI)

The backend executes a dual-model pipeline: an XGBoost Classifier to categorize blood pressure readings into AHA Stages, and an Isolation Forest model to flag statistical anomalies.

5.1 Model Evaluation Metrics (Data Science Baseline)

To satisfy core Data Science requirements before cloud deployment, the XGBoost classification model was rigorously evaluated on a holdout test set (30% split). Given the clinical context where false negatives (missing a hypertensive crisis) are highly dangerous, the model was optimized specifically for the **F1-Score** and **Recall** metrics on the minority hypertensive classes.

- **Global Accuracy:** 94.2%
- **Precision (Hypertensive Class):** 0.91
- **Recall (Hypertensive Class):** 0.95
- **F1-Score (Hypertensive Class):** 0.93
- **ROC-AUC:** 0.97

These baseline metrics show the model possesses high clinical validity prior to being wrapped in the FastAPI delivery mechanism.

5.2 Dual-Layer SHAP Translation and Confidence Scoring

To resolve the "black box" problem, predictions are passed through a `shap.TreeExplainer`. The mathematical SHAP values are extracted and dynamically translated into an English clinical summary.

To build trust in the results, the XGBoost model uses the `multi:softprob` objective during training. It outputs a probability array instead of a hard classification. If the model is only 52% confident, we show this to the clinician. This helps prevent automation bias.

Example System Output via Swagger:


```
{
  "prediction": "Elevated",
  "confidence_score": 0.8912,
  "shap_attributions": {
    "systolic": 1.102,
    "diastolic": -0.405,
    "map": 0.884,
    "pulse_pressure": 0.601,
    "is_night": -0.05,
    "is_dipper": -0.10
  },
  "clinical_summary": "Patient Risk Rating: Elevated (Model Confidence: 89.1%). Primary
driver: Systolic (+1.102 impact on risk score), followed by Map (+0.884 impact)."
}
```


5.3 Human-in-the-Loop (HITL) Feedback and MLOps Ticketing

To ensure continuous model improvement and safe clinical deployment, the architecture incorporates a "Human-in-the-Loop" (HITL) feedback mechanism. If a clinician's professional medical evaluation contradicts the AI's prediction (e.g., identifying a false negative), the clinician submits a "Clinical Override."

To bridge the communication gap between frontline medical staff and the Data Science (DS) team, this override triggers an automated backend webhook to the hospital's IT Service Management (ITSM) platform (e.g., Jira or ServiceNow).

Example Automated ITSM Ticket Payload (Closed Loop):


```
{
  "ticket_id": "MLOPS-842",
  "ticket_type": "Model Degradation Alert",
  "status": "Required feedback implemented - Model Retrained",
  "priority": "High",
  "routing": "Data Science & MLOps Team",
  "telemetry_reading": {"systolic": 135, "diastolic": 85},
  "ai_prediction": "Normal",
  "clinician_override": "Stage 1 Hypertension",
  "clinical_notes": "Patient exhibited sustained isolated systolic hypertension not
captured by the current XGBoost decision thresholds."
}
```


By continuously capturing these ground-truth corrections, the Data Science team can actively monitor the "Model Drift" metric. If the clinical override rate exceeds an acceptable medical safety threshold (e.g.-->3%), the DS team utilizes the newly corrected labels to execute the `POST /api/modeling/train` endpoint. Once the pipeline successfully retrains and deploys the updated model, the Jira ticket is automatically transitioned to **"Required feedback implemented,"** instantly notifying the clinician that their medical expertise has successfully updated the AI.

5.4 System Observability: Cloud Logging and Prometheus Metrics

Enterprise medical applications require real-time observability to ensure HIPAA compliance and architectural health.

- **Cloud Access Logging:** The FastAPI application utilizes native Python structured logging. Deployed on the Render cloud infrastructure, `stdout` and `stderr` streams are automatically aggregated, creating an immutable, timestamped audit log of all access requests (including `403 Forbidden` and `401 Unauthorized` interception events) to satisfy HIPAA auditing requirements.
- **User Behavior and Data Lake Analytics:** We capture raw user behavior and API request logs. While standard logs are good for debugging, they are hard to analyze at scale. We can stream these JSON logs directly into a data warehouse like Google BigQuery or a similar data lake. This allows the data science team to run heavy SQL analytics on user behavior over time without slowing down the primary PostgreSQL database.
- **Prometheus MLOps Metrics (Foundational Plumbing):** The backend integrates `prometheus-fastapi-instrumentator` to automatically generate a live telemetry feed in the industry-standard OpenMetrics format, exposed via the `GET /api/metrics` endpoint.

Future State: Pull-Based Metrics Aggregation

The API has basic logging and monitoring in place. We use a **pull-based architecture** for metrics. Pushing metrics can add network latency and cause crashes if the monitoring server goes offline. In future iterations, a metrics aggregator (such as Grafana Cloud, Datadog, or a dedicated Prometheus server) will be configured to "scrape" this endpoint every 10 to 15 seconds. This decoupled design ensures the telehealth backend remains highly performant while seamlessly allowing hospital IT teams to generate real-time graphical dashboards tracking Machine Learning inference latency, request volume, and overall system health.

Automated Alerting and Incident Response (On-Call Schedule)

Dashboards provide passive monitoring. We also need active alerting when failures happen. Once the Prometheus server is fully aggregating the `/metrics` endpoint, it will be paired with an Alertmanager. The Alertmanager evaluates critical Service Level Indicators (SLIs) in real-time. If an anomaly is detected, such as the HTTP 500 error rate exceeding 1%, or the p95 model inference latency spiking above 500ms for more than 3 consecutive minutes, an automated webhook is triggered. This webhook routes directly to an incident management platform (e.g., PagerDuty or Opsgenie). Based on the hospital IT's predefined escalation schedule, the system will immediately page the on-call DevOps Site Reliability Engineer (SRE) via SMS and Slack. This helps us fix pipeline failures before they affect patient care.

6. Scaling, Load Testing, API Fallbacks, and Rate Limiting

To address inquiries regarding system scalability and whether the encryption/RBAC mechanisms introduce performance bottlenecks, the API was subjected to automated load testing using **Locust**.

6.1 Rate Limiting and Device Fingerprinting (DDoS Prevention)

We implemented `slowapi` rate limiting to keep the system stable under load. We use **Heuristic Device Fingerprinting** to stop attackers from spoofing IP addresses. Rather than relying solely on IP addresses, the backend computes a SHA-256 cryptographic hash of the incoming IP, User-Agent, and Accept-Language headers. This creates a basic firewall against credential stuffing and malicious telemetry spam.

- **/api/auth/login:** Capped at 10 requests per minute to stop dictionary attacks.
- **/api/modeling/explain:** Capped at 60 requests per minute to prevent Machine Learning compute exhaustion (Inference DDoS).

Spike Testing Verification: To prove the resilience of this firewall, the system was subjected to a targeted "Spike Test" using Locust (200 concurrent users spawning instantly). As documented in the attached **Appendix B (Spike Testing Report)**, the firewall correctly intercepted the massive surge, yielding a 95%+ rejection rate with HTTP `429 Too Many Requests` errors.

Cost-Aware Scaling Strategy (Pod Caps): A high failure rate on a load test usually points to a bottleneck. However, these HTTP 429 rejections are an intentional design choice, not a bug. In a cloud environment like Render, automated horizontal scaling incurs direct financial compute costs. To prevent a billing runaway scenario during a DDoS attack or hardware malfunction, the infrastructure is budget-capped to a maximum of three concurrent web instances (<3 pods). Because the system cannot scale infinitely by design, it deliberately relies on the rate limiter to aggressively reject excess traffic at the edge network, thereby protecting both the core ML processing engine from crashing and the hospital from high cloud costs.

6.2 Locust Load Testing and Horizontal Scaling (Render)

While load testing is critical, true enterprise viability requires an automated scaling strategy. The telehealth architecture achieves this through a stateless design natively supported by the Render cloud platform:

- **Stateless API Design:** By relying on JSON Web Tokens (JWTs) for authorization and executing the XGBoost model entirely in-memory, the FastAPI application retains no local user state.
- **Render Automated Scaling:** Because the architecture is stateless, the Render cloud platform can easily scale the application horizontally. During sudden traffic spikes, Render's built-in load balancers automatically spin up duplicate containerized instances of the API, instantly distributing the incoming telehealth telemetry across multiple servers without requiring manual IT intervention.

To prove the baseline efficiency of a single API node before horizontal scaling occurs, the system was subjected to automated load testing using **Locust**. Simulating high-concurrency clinical access yielded the following metrics, proving the API can easily absorb traffic spikes:

Endpoint Tested	Simulation Goal	Avg Latency (ms)	p95 Latency (ms)	Server Failure Rate
POST /api/auth/login	OAuth2 JWT Generation	12 ms	18 ms	0%
POST /api/clinical/patients	Database Encryption	28 ms	45 ms	0%

	(pgp_sym_encrypt)			
POST /api/modeling/explain	XGBoost Inference + SHAP Extraction	85 ms	115 ms	0%
POST /api/modeling/explain	Malicious Unauthenticated Rejection	2 ms	4 ms	0% (Clean 401 Rejections)

Conclusion: The security implementations and Machine Learning pipelines introduce negligible overhead and are well within the acceptable medical industry threshold (<200ms) for real-time telemetry processing. A physical copy of the interactive Locust HTML report is attached to this submission as Appendix A.

6.3 Caching Strategy and CDN Omission

Standard web apps use Content Delivery Networks (CDNs) to reduce latency. Our telehealth API bypasses CDN caching for all endpoints. The system processes sensitive Protected Health Information (PHI). Caching responses at external edge nodes creates HIPAA compliance risks and serves stale data.

Instead, performance optimization relies exclusively on **In-Memory Model Caching** within the FastAPI backend. Upon the first API request, the XGBoost `.pkl` binary is loaded directly into RAM using a Singleton pattern. We cache the model in memory instead of reading from disk for every request. This removes processing bottlenecks and helps the ML pipeline achieve <100ms inference times.

Architectural Trade-offs:

While this caching strategy maximizes data security and inference speed, it requires accepting two specific trade-offs:

1. Geographic Latency (No CDN): Because all traffic must route directly to the central origin server to bypass edge caching, international clinicians will experience higher network latency (ping) than domestic users. This trade-off is accepted because HIPAA compliance overrides geographic speed optimizations.

2. RAM Consumption (In-Memory Caching): Pinning machine learning models in memory permanently consumes container RAM. If future iterations of this pipeline introduce larger models, the system will quickly exceed standard cloud memory limits, forcing the organization to procure more expensive, high-memory server tiers to prevent "Out of Memory" (OOM) crashes.

6.4 Manual API Execution (cURL Fallbacks)

In the event that the interactive Swagger UI (`/docs`) is temporarily unreachable, or if clinical IT staff wish to test the API programmatically, the system can be accessed directly via command-line `cURL` commands against the cloud-deployed backend.

1. Authenticate and Extract JWT (Administrator)

```
curl -X POST "https://telehealth-hypertension-analytics.onrender.com/api/auth/login" \
     -H "Content-Type: application/x-www-form-urlencoded" \
```

```
     -d "username=superadmin&password=superadmin123"
```

(Note: For all subsequent requests, replace `YOUR_JWT_TOKEN` with the `access_token` string returned from Step 1).

2. Execute XGBoost Prediction and SHAP Translation

```
curl -X POST "https://telehealth-hypertension-analytics.onrender.com/api/modeling/explain"
\
     -H "Authorization: Bearer YOUR_JWT_TOKEN" \
     -H "Content-Type: application/json" \
     -d '{"systolic": 135, "diastolic": 85, "map": 101.6, "pulse_pressure": 50, "is_night":
0, "is_dipper": 0}'
```

3. Securely Register a Patient (Triggers Database Encryption)

```
curl -X POST "https://telehealth-hypertension-analytics.onrender.com/api/clinical/patients"
\
     -H "Authorization: Bearer YOUR_JWT_TOKEN" \
     -H "Content-Type: application/json" \
     -d '{"first_name": "John", "last_name": "Doe", "ssn_last_four": "1234",
"contact_email": "johndoe@example.com", "date_of_birth": "1980-01-01"}'
```

4. Execute Machine Learning Retraining Pipeline (Admin Only)

```
curl -X POST "https://telehealth-hypertension-analytics.onrender.com/api/modeling/train" \
     -H "Authorization: Bearer YOUR_JWT_TOKEN"
```

6.5 Infrastructure Cost Estimation

When transitioning from an academic Proof of Concept (PoC) to a live medical environment, cloud pricing shifts from free tiers to production SLAs. Free tiers (which are utilized for this capstone deployment) suffer from "cold starts", where the server spins down after 15 minutes of inactivity. They also lack horizontal scaling.

To guarantee the <100ms real-time latency required for clinical use and to support up to 3 concurrent load-balanced pods, the architecture must run on paid production tiers. Here is the cost comparison:

Component	Service Used	Academic PoC (Free)	Production Enterprise (Monthly)
Frontend UI	Vercel	$0 (Hobby Tier)	$20 (Pro Tier)
API Backend	Render Web Service	$0 (Single Free Pod, sleeps)	$25 - $75 (Always-on, max 3 pods)
Database	Render Managed PostgreSQL	$0 (Expires in 30 days)	$20 (Persistent, Backups)

Data Lake	Google BigQuery	$0 (Sandbox)	< $5 (Pay-per-query)
Total Cost		**$0 / month**	**~$45 - $120 / month**

---

7. Error Handling, Debugging, and Troubleshooting

During the initial testing and deployment phases, several coding, data processing, and efficiency challenges were identified and resolved to ensure the model's readiness for a real-world clinical application.

7.1 Coding Errors and API Debugging

Challenge: When initially deploying the DDoS prevention firewall, the slowapi rate limiter caused fatal HTTP 500 crashes across all secured endpoints.

Resolution: Debugging the FastAPI traceback revealed that slowapi requires the raw ASGI Request object to dynamically extract client IP addresses. The API endpoint signatures were entirely refactored to inject the astapi.Request object alongside the JWT authentication payload. This resolved the crashes and successfully restored the rate-limiting functionality without breaking the interactive Swagger UI.

7.2 Data Processing Inconsistencies

Challenge: During the initial training phase, the synthetic telemetry generator produced data that was statistically sound but physiologically impossible (e.g., Diastolic pressures exceeding Systolic pressures, or fatal Mean Arterial Pressures). Training the XGBoost model on this raw data severely degraded its clinical accuracy.

Resolution (Pre-processing Adjustment): A deterministic clinical bounding script was introduced into the data pre-processing pipeline. It enforces strict physiological rules (e.g., Systolic must be > Diastolic, and MAP is strictly bounded between 70–110 mmHg). By correcting these inconsistencies before they reached the model, the false-positive rate dropped significantly, directly improving the Precision and Recall metrics.

7.3 Efficiency and Performance Improvements

Challenge: Initial load testing revealed that reading the XGBoost .pkl binary file from the cloud server's hard drive for every incoming prediction request caused significant disk I/O bottlenecks, pushing inference latency above 400ms.

Resolution: To enhance efficiency, the backend architecture was refactored to utilize a Singleton pattern for in-memory model caching. Upon the first API request, the model is loaded directly into the server's RAM. This efficiency improvement bypassed the disk I/O entirely, dropping average model inference latency to <100ms, as verified by the final Locust performance metrics.

---

8. Future State: Agentic AI Workflows

While the current architecture utilizes traditional deterministic machine learning (XGBoost) for clinical predictions, future iterations of this system propose integrating **Agentic AI Workflows** to drastically reduce administrative and operational toil.

Crucially, to maintain strict HIPAA compliance and avoid FDA Software as a Medical Device (SaMD) regulatory triggers, these Large Language Model (LLM) agents will *not* be placed in the critical diagnostic path. Instead, they will target "high-return, low-risk" backend operations:

- **Automated MLOps Triage Agent:** Currently, if Prometheus detects data drift, a Data Scientist must manually query the database to investigate. In the future state, an autonomous agent will intercept the alert, query the SQL database to calculate the variance, and draft a complete root-cause analysis directly into a Jira ticket. This condenses hours of manual data-wrangling into a single "Approve Retrain" click for the engineering team.
- **Security Audit Agent:** To manage the massive volume of logs generated by the `slowapi` rate limiter, a scheduled agent will run nightly to analyze intercepted `401 Unauthorized` and `429 Too Many Requests` traffic. It will autonomously summarize potential credential stuffing or botnet attacks into a morning threat brief for IT administrators.

By pointing LLM agents at backend system maintenance rather than direct patient diagnosis, the architecture can safely leverage advanced AI reasoning without compromising the deterministic safety of the clinical predictions.

References

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794. https://doi.org/10.1145/2939672.2939785

Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation forest. *2008 Eighth IEEE International Conference on Data Mining*, 413–422. https://doi.org/10.1109/ICDM.2008.17

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30, 4765–4774.

Saltz, J., & Krasteva, I. (2022). Current approaches for executing big data science projects: A systematic literature review. *PeerJ Computer Science*, 8, e942. https://doi.org/10.7717/peerj-cs.942

Spector, A., Norvig, P., Wiggins, C., & Wing, J. M. (2022). *Data science in context: Foundations, challenges, opportunities*. Cambridge University Press. https://doi.org/10.1016/9781009272230

Sylligardos, M., Paul, A., Mercer, B., & Palpanas, T. (2023). Choose wisely: An extensive evaluation of model selection for anomaly detection in time series. *Proceedings of the VLDB Endowment*, 16(11), 3326–3339. https://doi.org/10.14778/3611479.3611536

Tsao, C. W., Aday, A. W., Almarzooq, Z. I., Anderson, C. A., Arora, P., Avery, C. L., ... & Martin, S. S. (2023). Heart disease and stroke statistics—2023 update: A report from the American Heart Association. *Circulation*, 147(8), e93–e621. https://doi.org/10.1161/CIR.0000000000001123

U.S. Department of Health and Human Services. (2013). *Modifications to the HIPAA Privacy, Security, Enforcement, and Breach Notification Rules under the Health Information Technology for Economic and Clinical Health Act*. Office for Civil Rights.

Whelton, P. K., Carey, R. M., Aronow, W. S., Casey, D. E., Collins, K. J., Himmelfarb, C. D., ... & Wright, J. T. (2018). 2017 ACC/AHA/AAPA/ABC/ACPM/AGS/APhA/ASH/ASPC/NMA/PCNA guideline for the prevention, detection, evaluation, and management of high blood pressure in adults. *Journal of the American College of Cardiology*, 71(19), e127–e248. https://doi.org/10.1016/j.jacc.2017.11.006